

Презентация по лабораторной работе 5

Имитационное моделирование

Нджову Нелиа

Цель работы

Цель этой лабораторной работы - моделирования Аппарат сетей петри.

1 Код модели

Задание

- 1 Код модели
- 2 Базовые эксперименты

- 1 Код модели
- 2 Базовые эксперименты
- 3 Анимация процесса

- 1 Код модели
- 2 Базовые эксперименты
- 3 Анимация процесса
- 4 Итоговый отчёт

Сеть Петри есть математический аппарат для моделирования дискретных систем. в которых процессы происходят параллельно и асинхронно. Она позволяет выявлять такие проблемы, как взаимная блокировка (deadlock). Графически она представляется как двудольный ориентированный граф двух типов вершин:

- Положения (Places) суть пассивные элементы, описывающие состояние системы. Внутри позиции могут находиться фишки (tokens, неотрицательное целое число, указывающее на количество ресурсов)

Сеть Петри есть математический аппарат для моделирования дискретных систем. в которых процессы происходят параллельно и асинхронно. Она позволяет выявлять такие проблемы, как взаимная блокировка (deadlock). Графически она представляется как двудольный ориентированный граф двух типов вершин:

- Позиции (Places) суть пассивные элементы, описывающие состояние системы. Внутри позиции могут находиться фишки (tokens, неотрицательное целое число, указывающее на количество ресурсов)
- Переходы (Transitions) суть активные элементы, описывающие события или действия системы. Они могут срабатывать, изменяя состояние модели.

Сеть Петри есть математический аппарат для моделирования дискретных систем. в которых процессы происходят параллельно и асинхронно. Она позволяет выявлять такие проблемы, как взаимная блокировка (deadlock). Графически она представляется как двудольный ориентированный граф двух типов вершин:

- Позиции (Places) суть пассивные элементы, описывающие состояние системы. Внутри позиции могут находиться фишки (tokens, неотрицательное целое число, указывающее на количество ресурсов)
- Переходы (Transitions) суть активные элементы, описывающие события или действия системы. Они могут срабатывать, изменяя состояние модели.
- Дуги соединяют позиции с переходами и показывают, какие ресурсы нужны для события и какие появляются после него.

Правило срабатывания перехода: Переход может сработать, если каждая его входная позиция содержит хотя бы одну фишку. При срабатывании из входных позиций фишки удаляются, а в выходные добавляются. Этот процесс меняет маркировку сети, то есть её текущее состояние.

Сети Петри бывают детерминированными (описываются дифференциальными уравнениями) и стохастическими (используют алгоритм Гиллеспи, где время и выбор перехода определяются случайным образом).

Задача «Обедающие философы»

Это классическая задача, иллюстрирующая проблему синхронизации в параллельных системах.

Постановка задачи: За круглым столом сидят N философов (обычно 5). Перед каждым тарелка с едой. Между каждыми двумя соседними философами лежит одна вилка. Всего вилок столько же, сколько философов.

Правила: Чтобы поест, философу нужны две вилки — левая и правая. Если вилок нет, философ ждёт. Поев, философ кладёт вилки обратно и возвращается к размышлениям.

Проблема взаимной блокировки (deadlock): Если все философы одновременно возьмут левую вилку, то каждый будет держать одну вилку и ждать правую, которая занята соседом. В результате никто не может есть — система замирает. Это и есть deadlock.

Решение (арбитр): Чтобы предотвратить deadlock, вводится арбитр (официант), который разрешает есть не более чем $N-1$ философам одновременно. Это гарантирует, что хотя бы один философ всегда сможет взять обе вилки, и система не блокируется.

1. Код модели

Я сначала создала файл `src/DiningPhilosophers.jl`. Потом скопировала заданный скрипт в него. Скрипт содержит основные функции модели. Я запускала и все работало хорошо(рис.1)

1. Код модели

```
dining_philosophers.jl × DiningPhilosophers.jl ×
module DiningPhilosophers

using OrdinaryDiffEq
using Plots
using DataFrames
using Random, LinearAlgebra

export build_classical_network, build_arbiter_network
export simulate_ode, simulate_stochastic
export detect_deadlock, plot_marking_evolution

# Определение простой структуры PetriNet
struct PetriNet
    n_places::Int
    n_transitions::Int
    incidence::Matrix{Int}
    place_names::Vector{Symbol}
    transition_names::Vector{Symbol}
end

function PetriNet(
    n_places,
    n_transitions;
    place_names = Symbol[],
    transition_names = Symbol[],
```

2. Базовые эксперименты

Я создала файл `scripts/dining_philosophers.jl`. Потом скопировала заданный скрипт в него. Скрипт выполняет основное моделирование и сравнение двух вариантов сети Петри: классическая модель (без арбитра), в которой возможна взаимная блокировка (deadlock). И модифицированная модель с арбитром, которая должна предотвращать deadlock. Я преобразовала код в литературный стиль и запускала.(рис.2)

2. Базовые эксперименты

```
dining_philosophers.jl × DiningPhilosophers.jl ×
using DrWatson
@quickactivate "project"
include(scrdir("DiningPhilosophers.jl"))
using .DiningPhilosophers
using DataFrames, CSV, Plots

N = 5
tmax = 50.0

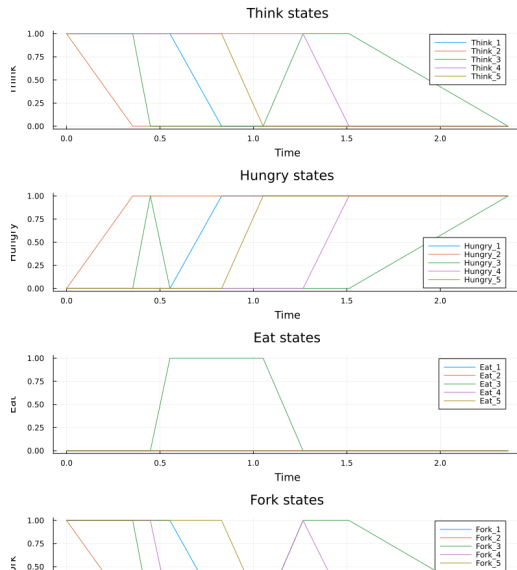
println("=== Классическая сеть (без арбитра) ===")
net_classic, u0_classic, _ = build_classical_network(N)
df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
CSV.write(datadir("dining_classic.csv"), df_classic)
dead = detect_deadlock(df_classic, net_classic)
println("Deadlock обнаружен: $dead")
plot_classic = plot_marking_evolution(df_classic, N)
savefig(plotsdir("classic_simulation.png"))

println("\n=== Сеть с арбитром ===")
net_arb, u0_arb, _ = build_arbiter_network(N)
df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
CSV.write(datadir("dining_arbiter.csv"), df_arb)
dead_arb = detect_deadlock(df_arb, net_arb)
```

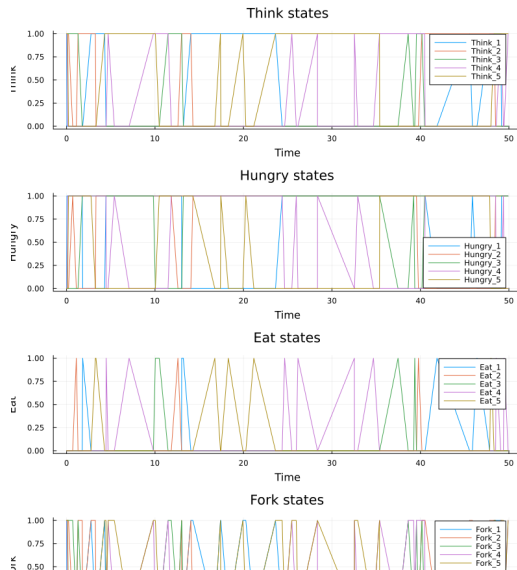

2. Базовые эксперименты

После запуск, я получила две графики `classic_simulation.png` и `arbiter_simulation.png`. Мы увидим четыре панели (Think, Hungry, Eat, Fork) с динамикой для каждого философа. В классическом симуляторе произошла взаимоблокировка, но в режиме арбитра этого не произошло (рис 3 и 4)

2. Базовые эксперименты

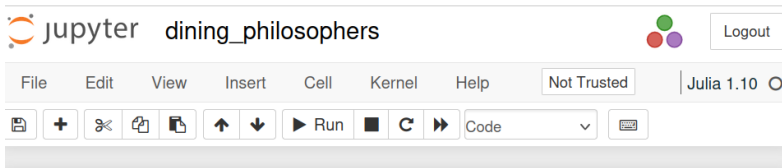


2. Базовые эксперименты



2. Базовые эксперименты

Я генерировала файл jupyter, запускала все ячейки и всё работал хорошо.(рис.5)



Базовые эксперименты

Зачем нужен Скрипт выполняет основное моделирование и сравнение двух вариантов сети Петри: классическая модель (без арбитра), в которой возможна взаимная блокировка (deadlock) и модифицированная модель с арбитром, которая должна предотвращать deadlock.

```
In [1]: using DrWatson
@quickactivate "project"
include(sourcedir("DiningPhilosophers.jl"))
using .DiningPhilosophers
using DataFrames, CSV, Plots
```

2. Базовые эксперименты

Я изменила количество философов в эксперименте, я использовала $N = 2$ и $N = 10$. И в обоих экспериментах тупик возник в классическом моделировании, но не в произвольном моделировании(рис.6)

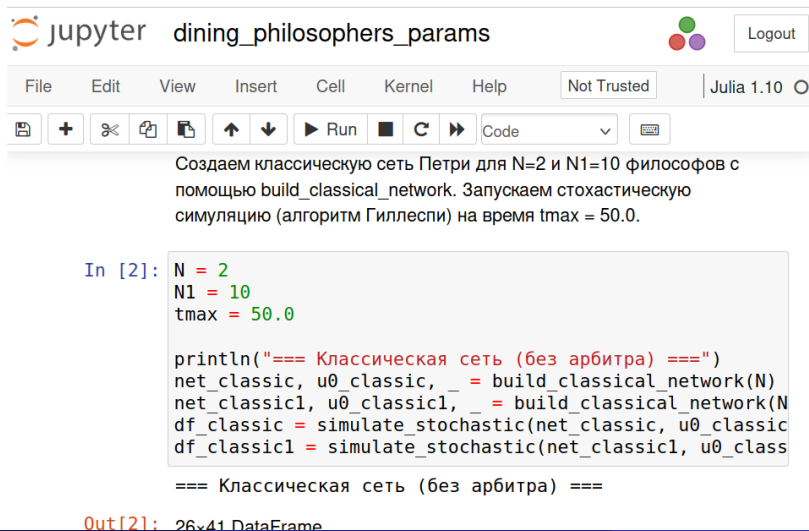
```
nelianjovu@Nebby:~/work/study/2026-1/2026-1==study--simulationmod/2026-1--study--simulationmod/labs/lab05/project$ julia --project=. scripts/dining_philosophers_params.jl
=== Классическая сеть (без арбитра) ===
Deadlock обнаружен(N=2): true
Deadlock обнаружен(N=10): true

=== Сеть с арбитром ===
Deadlock обнаружен(N=2): false
Deadlock обнаружен(N=10): false
```

Рисунок 6: изменения количества философов

2. Базовые эксперименты

Я генерировала файл jupyter, запускала все ячейки и всё работал хорошо.(рис.7)



Создаем классическую сеть Петри для $N=2$ и $N1=10$ философов с помощью `build_classical_network`. Запускаем стохастическую симуляцию (алгоритм Гиллеспи) на время $t_{max} = 50.0$.

```
In [2]: N = 2
        N1 = 10
        tmax = 50.0

        println("=== Классическая сеть (без арбитра) ===")
        net_classic, u0_classic, _ = build_classical_network(N)
        net_classic1, u0_classic1, _ = build_classical_network(N1)
        df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
        df_classic1 = simulate_stochastic(net_classic1, u0_classic1, tmax)

        === Классическая сеть (без арбитра) ===
```

Out[2]: 26x41 DataFrame

3. Анимация процесса

Я создала файл `scripts/dining_philosophers_animation.jl`. Потом скопировала заданный скрипт в него. Скрипт используется для наглядной демонстрации динамики работы сети Петри во времени. Анимация позволяет увидеть, как меняется маркировка (фишки) в каждой позиции, и особенно наглядно показывает возникновение deadlock в классической модели(рис.8)

3. Анимация процесса

```
dining_philosophers_params.jl  dining_philosophers_animate.jl
# ## Анимация процесса
# **Зачем нужен:** Для наглядной демонстрации динамики работы сети
# Петри во времени и Анимация позволяет увидеть, как меняется
# маркировка (фишки) в каждой позиции, и особенно наглядно показывает
# возникновение deadlock в классической модели.

using DrWatson
@quickactivate "project"
include(sourcedir("DiningPhilosophers.jl"))
using .DiningPhilosophers
using Plots, Random

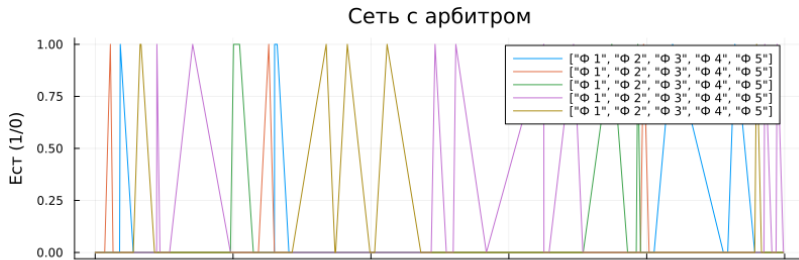
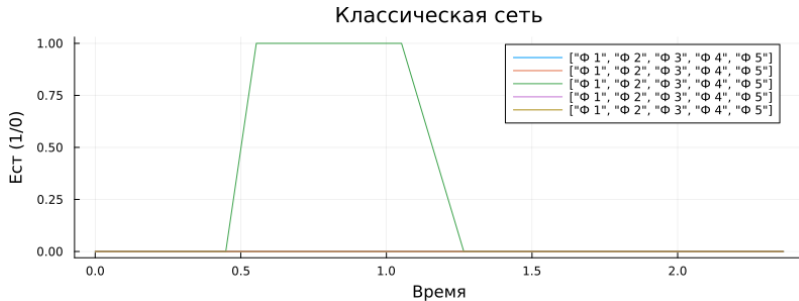
# Берёт классическую сеть (для малого числа философов (N=3) для
# упрощения визуализации). Запускает стохастическую симуляцию на tmax
# = 30.0.

N = 3
tmax = 30.0
net, u0, names = build_classical_network(N)
```


4. Итоговый отчёт

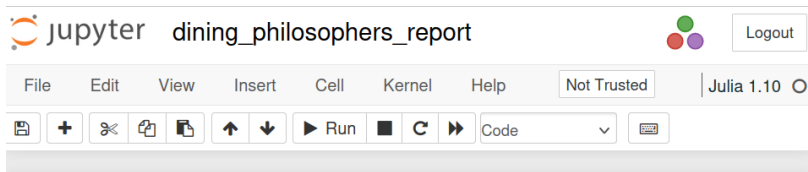
Я создала файл `scripts/dining_philosophers_report.jl`. Потом скопировала заданный скрипт в него. Скрипт используется для сравнительного анализа двух моделей (с арбитром и без) по одному ключевому показателю — числу философов, находящихся в состоянии «Ест» (`Eat_i`). Скрипт генерирует сводный график.(рис.9)

4. ИТОГОВЫЙ ОТЧЁТ



4. Итоговый отчёт

Я преобразовала код в литературный стиль, генерировала файл jupyter и выполнила все ячейки(рис.10)



Итоговый отчёт

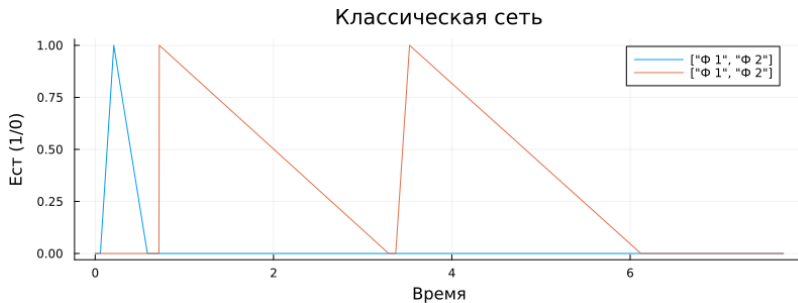
Зачем нужен: Для сравнительного анализа двух моделей (с арбитром и без) по одному ключевому показателю — числу философов, находящихся в состоянии «Ест» (Eat_i). Скрипт генерирует сводный график.

```
In [1]: using DrWatson
@quickactivate "project"
using DataFrames, CSV, Plots
```

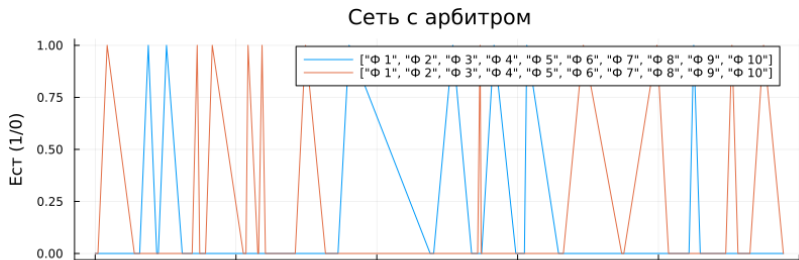
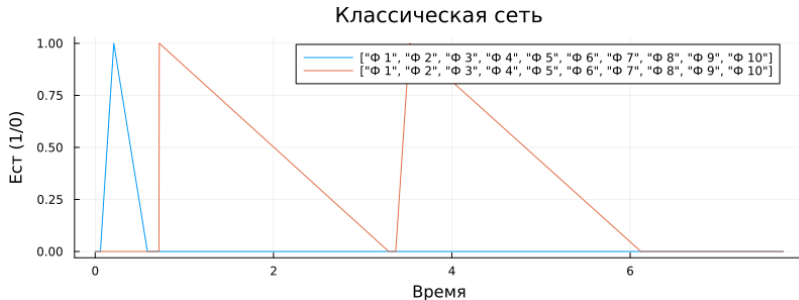
4. Итоговый отчёт

Я изменила скрипт, чтобы он работал при $N = 2$ и $N = 10$ (рис. 11 и 12)

4. ИТОГОВЫЙ ОТЧЁТ

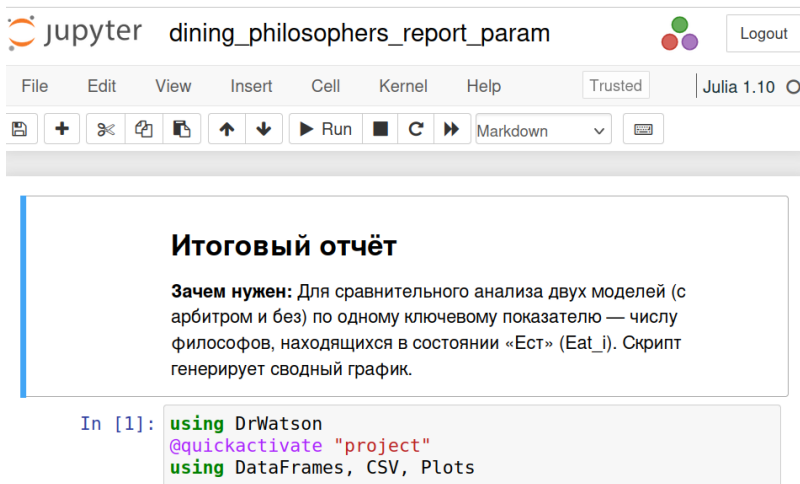


4. ИТОГОВЫЙ ОТЧЁТ



4. Итоговый отчёт

Я преобразовала код в литературный стиль, генерировала файл jupyter и выполнила все ячейки(рис.13)



The screenshot shows a Jupyter Notebook interface. At the top, the title bar reads 'jupyter dining_philosophers_report_param' with a 'Logout' button. Below this is a menu bar with 'File', 'Edit', 'View', 'Insert', 'Cell', 'Kernel', and 'Help'. To the right of the menu bar are 'Trusted' and 'Julia 1.10' status indicators. A toolbar contains icons for saving, adding cells, undo, redo, and running code. The main content area displays a report titled 'Итоговый отчёт' (Final Report). The report text states: 'Зачем нужен: Для сравнительного анализа двух моделей (с арбитром и без) по одному ключевому показателю — числу философов, находящихся в состоянии «Ест» (Eat_i). Скрипт генерирует сводный график.' (Why it's needed: For comparative analysis of two models (with and without an arbitrator) by one key indicator — the number of philosophers in the 'Eat' state (Eat_i). The script generates a summary chart.) Below the report text, a code cell is shown with the following Julia code:

```
In [1]: using DrWatson
@quickactivate "project"
using DataFrames, CSV, Plots
```

Выполняя эту лабораторную работу, научилась моделировать аппарат сетей Петри.